DISTINCT RUNS IN STANDARD-ORDER COMPOSITIONS: AN EXACT BINARY-LENGTH FORMULA FOR OEIS A351015

DANIYAL KADIRBEKOV

Abstract. Let $a(n)$ be the smallest nonnegative integer k such that the k-th composition in standard order has exactly n distinct runs; this is OEIS A351015. Our main result is an exact formula for the binary length of $a(n)$. Representing a run of l copies of a part v by the type (v, l), whose block in the standard binary encoding has length vl, we prove that

$$\operatorname{bitlength}\big(a(n)\big) = \sum_{j=1}^{n} s(j),$$

where s is OEIS A061017, the multiplication table of the positive integers sorted with multiplicity. The proof shows that the induced lower bound is always attained and describes the length-minimizing compositions exactly. Turning the formula into an exact lexicographic dynamic program pins down $a(n)$ itself; as a consequence we extend the sequence, obtaining for instance

$$a(20) = 5\,090\,566\,682\,637\,483\,446\,698\,432\,134\,911.$$

The formula also gives $\log_2 a(n) \sim n^2/(2\log n)$.

1. Introduction

How early must one look, among the integer compositions, to find one with n different kinds of run? Ordered by the standard bijection with the nonnegative integers, the compositions turn this into a question about integers: what is the least k whose composition has exactly n distinct runs? As a function of n, the answer is the sequence studied here, OEIS A351015.

We recall the standard order. If the binary expansion of $k > 0$ is read from left to right, each 1 starts a new part of size 1 and each subsequent 0 increases the current part by 1; equivalently a part v is encoded by the word 10^{v-1}, and 0 encodes the empty composition. This bijection between the nonnegative integers and the finite compositions is OEIS A066099 [2]. A *run* is a maximal block of equal parts, so a run of l copies of the value v has type (v, l), and two runs are distinct exactly when their types differ. For instance,

Date: July 2026.

2020 *Mathematics Subject Classification.* Primary 11B83; Secondary 05A05, 11N37, 68R15.

Key words and phrases. integer composition, standard order, run, distinct runs, OEIS A351015, divisor function, combinatorics on words, asymptotic enumeration.

$27 = (11011)_2$ encodes the composition $(1, 2, 1, 1)$, whose runs (1), (2), $(1, 1)$ have the three distinct types $(1, 1)$, $(2, 1)$, and $(1, 2)$; no smaller index has three distinct runs, so $27 = a(3)$. In general, following Gus Wiseman [1],

$$a(n) = \min\{k \geq 0 : \text{ the standard composition of } k \text{ has } n \text{ distinct runs}\}.$$

When it was introduced, only $a(0)$ through $a(7)$ were known, obtained by a direct search over k; the entry asks for a formula or a general construction, and none was available. A direct search is in any case quickly defeated by the growth of the values: as Section 4 records, already $a(10) = 4\,503\,885\,791$. The purpose of this note is to supply such a construction, replacing the linear scan over integers by a structural description of the length-minimizing compositions together with an exact finite optimization.

Runs in integer compositions have previously been studied through generating functions for restrictions on run length; see Wilf [9]. A close OEIS analogue is A350952 [5], which asks for the smallest integer whose binary expansion has a prescribed number of distinct runs. That problem has a two-letter alphabet and charges one bit per position. Here the part values are unbounded, and a run of length l in the value v has cost vl, producing a different extremal problem.

Several neighbouring OEIS sequences concern the same statistic on standard-order compositions. A351014 [7] gives the number of distinct runs of the n-th composition, of which A351015 is the sequence of first appearances; A351013 [6] counts the compositions of n whose runs are all distinct, and A351290 [8] lists the standard indices of such compositions. The related statistical question of the longest single run in a random composition is treated by Gafni [10]. None of these addresses the minimum-size (first-occurrence) problem or the length formula obtained here.

The starting point is a simple observation: the standard encoding turns a purely combinatorial constraint (the number of distinct runs) into an additive one (the binary length). This yields our main result, an exact formula for bitlength$(a(n))$ through OEIS A061017 (Theorem 2.2, Section 2). From it we obtain, in turn, a structural characterization of the length-minimizing compositions together with an exact algorithm for $a(n)$ (Section 3); the resulting values through $a(20)$, with a self-contained certificate for $a(10)$ (Section 4); and the asymptotic growth $\log_2 a(n) \sim n^2/(2 \log n)$ (Section 5). Throughout, the computation supports the theory but is nowhere used to prove it. Section 6 discusses significance and states an open problem.

2. The exact binary-length theorem

Let $s(1), s(2), \ldots$ denote OEIS A061017 [3]:

$$1, 2, 2, 3, 3, 4, 4, 4, 5, 5, 6, 6, 6, 6, 7, 7, \ldots.$$

Equivalently, each positive integer m occurs $d(m)$ times, where $d(m)$ is the number of positive divisors of m. This is precisely the multiset of products

vl over ordered positive pairs (v, l), sorted into nondecreasing order. Set

$$L(0) = 0, \qquad L(n) = \sum_{j=1}^{n} s(j) \quad (n \geq 1).$$

Lemma 2.1 (Rearrangement)**.** *A multiset of $N \geq 1$ labels can be placed in a linear order with no two equal labels adjacent if and only if its largest multiplicity M satisfies $M \leq N - M + 1$.*

Proof. The M copies of a most frequent label require at least $M - 1$ separating labels, so $M - 1 \leq N - M$; this proves necessity. For sufficiency we induct on N, the cases $N \leq 2$ being immediate. Let $N \geq 3$ with $2M \leq N+1$. Then at least two labels have positive multiplicity, since a single label would give $M = N$ and hence $2N \leq N + 1$, i.e. $N \leq 1$. Let x and y be labels of largest and second-largest positive multiplicity, and let c be the third-largest multiplicity, with $c = 0$ if fewer than three labels occur. Delete one copy of each of x and y. In the remaining multiset of $N - 2$ labels the largest multiplicity is $M' = \max(M - 1, c)$. If $M' = M - 1$ then $2M' = 2M - 2 \leq N - 1$. If $M' = c$, then $c \leq M$, with $c = M$ only when at least three labels attain M, whence $N \geq 3M$ and $2c = 2M \leq N - M \leq N - 1$; and if $c \leq M - 1$ then $2c \leq 2M - 2 \leq N - 1$. In every case $2M' \leq N - 1 = (N - 2) + 1$, so the induction hypothesis provides an arrangement w of the $N-2$ remaining labels with no two equal adjacent. Write x and y before w, choosing which one sits immediately before w so that it differs from the first label of w; this is possible because $x \neq y$, so at most one of them equals that label. The two new adjacencies are then x–y and (a label)–w_1, both between distinct labels, so the whole word has no two equal adjacent labels. □

Theorem 2.2. *For every $n \geq 0$,*

$$\operatorname{bitlength}\big(a(n)\big) = L(n) = \sum_{j=1}^{n} s(j),$$

where $\operatorname{bitlength}(0) = 0$.

Proof. The case $n = 0$ is immediate, so assume $n \geq 1$. A run of type (v, l) contains l parts of size v; it therefore contributes vl both to the sum of the composition and to the length of its standard binary encoding. Any composition with n distinct run types must contain at least one copy of each type, so its encoding length is at least the sum of the n smallest elements of the multiset of type costs, namely $L(n)$. This establishes the lower bound.

It remains to construct a composition attaining it. For $q \geq 0$, let

$$D(q) = \sum_{m \leq q} d(m) = \#\{(v, l) \in \mathbb{Z}_{>0}^2 : \ vl \leq q\},$$

the divisor summatory function (OEIS A006218 [4]). Choose q so that $D(q-1) < n \leq D(q)$, and put $r = n - D(q - 1)$. Take every run type (v, l) with $vl < q$, together with r run types satisfying $vl = q$: if $r < d(q)$, choose any r boundary types other than $(1, q)$; if $r = d(q)$, take all boundary types.

We show that these types can be ordered so that consecutive values v differ, applying Lemma 2.1 with each type's value v as its label. Let $N = n$ be the number of selected types and M the number of them with value 1. Take $q \geq 3$; the cases $q = 1, 2$ are immediate.

If $(1, q)$ is omitted, then $M = q - 1$, while each value $v \geq 2$ is carried by at most $\lfloor (q-1)/v \rfloor + 1 \leq q - 1$ selected types, so M is the largest label multiplicity. The $2q - 3$ pairs $(1, l)$ for $1 \leq l \leq q - 1$ and $(v, 1)$ for $2 \leq v \leq q - 1$ are distinct with product $< q$, so $D(q-1) \geq 2q - 3$; hence $N = n > D(q-1) \geq 2q - 3$, which gives $2M = 2(q-1) \leq N + 1$, that is, $M \leq N - M + 1$. If instead all boundary types are selected, then $M = q$ and $N = n = D(q)$, while each value $v \geq 2$ occurs in $\lfloor q/v \rfloor \leq q = M$ types, so M is again largest; adjoining $(1, q)$ and $(q, 1)$ to the same witnesses gives $D(q) \geq 2q - 1$, so $2M = 2q \leq N + 1$. In both cases Lemma 2.1 yields the required ordering.

Order the value labels with no equal adjacent labels and assign the selected multiplicities l to their corresponding positions. Adjacent run types now have different values, so their concatenation is a valid composition with exactly n distinct run types and total cost $L(n)$. The lower bound is therefore attained. □

An explicit arithmetic form of the theorem is useful in the sequel. If $D(q-1) < n \leq D(q)$, then

$$L(n) = \sum_{m<q} m\, d(m) + q\big(n - D(q-1)\big). \tag{2.1}$$

Remark. The length sequence $L(n) = \mathrm{bitlength}(a(n))$, beginning $0, 1, 3, 5, 8, 11, 15, 19, 23, 28, 33, \ldots$, is the sequence of partial sums of A061017. Theorem 2.2 therefore expresses one integer sequence, the binary lengths of A351015, exactly through another, the sorted multiplication table A061017.

3. Characterization of minimizers and an exact algorithm

The construction in the proof of Theorem 2.2 does more than attain the bound; it describes every minimizer.

Proposition 3.1. *Let $n \geq 1$, choose q with $D(q-1) < n \leq D(q)$, and set $r = n - D(q-1)$. A composition attains the minimum encoding length $L(n)$ among those with n distinct runs if and only if its runs are exactly the $D(q-1)$ types (v, l) with $vl < q$ together with r of the $d(q)$ types with $vl = q$, each used once, in some order with different values on consecutive runs.*

Proof. List the composition's runs with their types; the encoding length is the sum of their costs vl. A composition with n distinct runs uses n distinct types, each at least once, so this sum is at least the sum $L(n)$ of the n smallest costs, with equality if and only if (i) no type is used more than once, since a second occurrence adds a positive cost, and (ii) the n types used are precisely the n cheapest. The n cheapest costs are the $D(q-1)$

values $vl < q$, all of which must then occur, together with r of the $d(q)$ values $vl = q$; this is the stated multiset. Consecutive runs of a composition always carry different values, by maximality, and the proof of Theorem 2.2 produces an arrangement of this multiset with that property, so such compositions exist. □

Theorem 2.2 fixes the minimum word length and Proposition 3.1 the multiset of runs; the only remaining freedom is the boundary choice and the order of the runs. We determine the least admissible word by dynamic programming.

The search for $a(n)$ is thus a search for a least binary *word*. All admissible words have the same length $L(n)$, and for binary strings of equal length, comparison from the most significant bit (lexicographic order, with `0` < `1`) coincides with comparison of the integers they represent. Hence the least integer of length $L(n)$ is encoded by the lexicographically least admissible word, and by Theorem 2.2 that integer is $a(n)$.

For a run type (v, l), write

$$B(v, l) = \left(10^{v-1}\right)^{l}$$

for its binary block. Fix a candidate type set T (the forced types with one admissible boundary choice). If $S \subseteq T$ is the set of *unused* types and u is the value of the preceding run (with $u = 0$ before the first run, a sentinel distinct from every value), define

$$F(S, u) = \min_{\substack{(v,l)\in S \\ v\neq u}} B(v, l) \cdot F\big(S \setminus \{(v, l)\}, v\big),$$

where the product denotes concatenation and the minimum is lexicographic. The base case is $F(\varnothing, u) = \varepsilon$, the empty word; a state with no admissible transition is infeasible, with value $+\infty$ (larger than every finite word) so that it never attains a minimum.

Proposition 3.2. *For each candidate type set T, the word $F(T, 0)$ is the lexicographically least concatenation of the blocks of the types in T, each used once, in which no two consecutive runs share a value, and is infeasible only if no such concatenation exists. Consequently, over the admissible boundary choices T,*

$$a(n) = \min_{T} F(T, 0),$$

the minimum read as a binary integer, is the least term of length $L(n)$.

Proof. Fix T and prove by induction on $|S|$ that $F(S, u)$ is the lexicographically least concatenation of the blocks of S, each used once, whose first run has value different from u, with no two consecutive runs sharing a value; the case $S = \varnothing$ is immediate. For $|S| \geq 1$, every admissible word for (S, u) begins with the block $B(v, l)$ of some $(v, l) \in S$ with $v \neq u$ and continues with an admissible word for $(S \setminus \{(v, l)\}, v)$, and all such words share the common

length $\sum_{(v,l)\in S} vl$. For a fixed leading block the fixed prefix preserves lexicographic order, so the least continuation is $F(S \setminus \{(v,l)\}, v)$ by induction; taking the lexicographic minimum of $B(v,l) \cdot F(S \setminus \{(v,l)\}, v)$ over the admissible (v,l) (this is the recurrence) therefore returns the least admissible word. With $S = T$ and $u = 0$ this is the first assertion. Ranging over the admissible T, and using that for equal-length binary words lexicographic and numerical order agree, gives $a(n) = \min_T F(T, 0)$. □

For a fixed candidate set the dynamic program has $O(n\,2^n)$ states, one per pair (S, u) with $S \subseteq T$, and up to n transitions each, so $O(n^2 2^n)$ work. There are $\binom{d(q)}{r}$ admissible boundary choices, hence computing $a(n)$ costs

$$O\left(\binom{d(q)}{r} n^2\, 2^n\right).$$

This is exponential in n, but dramatically smaller than scanning all integers up to $a(n) = 2^{\Theta(n^2/\log n)}$ (Corollary 5.1).

4. New terms and a certificate for $a(10)$

Running the algorithm of Section 3 for $0 \le n \le 20$ yields Table 1.

Proposition 4.1. *The terms $a(n)$ for $0 \le n \le 20$ are those listed in Table 1; every entry through $a(9)$ agrees with OEIS A351015, and each is certified by an explicit length-minimizing composition.*

The values through $a(7)$ are the original entries; $a(8)$ and $a(9)$ were added to OEIS by the author, and $a(10)$ through $a(20)$ are new. Each term is produced together with the composition that Proposition 3.2 shows is optimal, so the table records certificates rather than the output of a search.

For $n = 10$, the ten smallest run-type costs are

$$1, 2, 2, 3, 3, 4, 4, 4, 5, 5,$$

whose sum is 33. Equality forces precisely the ten types (v,l) with $vl \le 5$. The dynamic program returns the binary word

$$\texttt{1000011000111001111010111111011111},$$

which encodes the composition

$$(5, 1, 4, 1, 1, 3, 1, 1, 1, 2, 2, 1, 1, 1, 1, 2, 1, 1, 1, 1, 1).$$

Its run types, in order, are

$$(5,1), (1,1), (4,1), (1,2), (3,1), (1,3), (2,2), (1,4), (2,1), (1,5).$$

They are all distinct, their total cost is 33, and exhaustive dynamic programming over every valid ordering proves that the displayed word is the least word of that length.

There is also a short direct argument. Write $A_j = (1,j)$ for $1 \le j \le 5$; the other five types are $(5,1), (4,1), (3,1), (2,2), (2,1)$, of which only $(2,2)$ and $(2,1)$ share a value. A least word cannot begin with an A_j, because

TABLE 1. The minimum binary length $L(n)$ and the term $a(n)$ of OEIS A351015, for $0 \le n \le 20$.

n	$L(n)$	$a(n)$
0	0	0
1	1	1
2	3	5
3	5	27
4	8	155
5	11	1 655
6	15	18 039
7	19	281 975
8	23	4 617 711
9	28	138 835 439
10	33	4 503 885 791
11	39	279 381 792 735
12	45	17 876 500 338 655
13	51	1 144 081 370 306 527
14	57	73 222 145 385 127 871
15	64	9 296 594 182 239 903 679
16	71	1 189 966 115 078 053 658 495
17	79	303 421 421 018 735 347 335 039
18	87	77 675 883 218 955 741 073 080 191
19	95	19 885 026 103 583 920 149 103 476 607
20	103	5 090 566 682 637 483 446 698 432 134 911

that gives the prefix `11`, whereas every other type begins `10`. Since no two value-1 types may be adjacent, the five A_j occupy one each of the five gaps after the other blocks, and the word alternates non-A, A, non-A, ..., ending in an A_j.

Greedy lexicographic minimization fixes both orders. Among the non-A blocks, $(5,1)$ = `10000`, $(4,1)$ = `1000`, and $(3,1)$ = `100` have a 0 in their third bit, whereas $(2,2)$ = `1010` and $(2,1)$ = `10` (followed by the 1 of the next block) have a 1 there; so the three value-≥ 3 blocks come first, and among them the longest (hence zero-richest) first, giving $(5,1),(4,1),(3,1)$. The two value-2 types are then fixed by comparing the two orderings, which agree on their first three bits and first differ at the fourth: placing $(2,2)$ before $(2,1)$ writes `1010`, while the reverse writes `10` followed by an all-ones block A_j, giving prefix `1011` (if $A_j = A_1$ is a single `1`, the fourth bit is instead the `1` beginning the next block, so the prefix is still `1011`); since `1010` < `1011`, the order is $(2,2),(2,1)$. At each of the first four A slots the smallest unused j reaches the following 0 soonest, placing A_1, A_2, A_3, A_4; the remaining A_5 fills the final slot. The resulting word is the one displayed above, and it is the unique lexicographic minimum, certifying $a(10) = 4\,503\,885\,791$ without a scan of smaller integers.

5. Asymptotic growth

Here and below, log denotes the natural logarithm. The classical divisor summatory estimate [11, §18.2] is

$$D(x) = x \log x + (2\gamma - 1)x + O(\sqrt{x}),$$

and partial summation gives

$$\sum_{m \le x} m\, d(m) = \tfrac{1}{2} x^2 \log x + O(x^2).$$

If $q = s(n)$, then $n \sim q \log q$, so $q \sim n/\log n$. Writing $r = n - D(q-1)$, Equation (2.1) has boundary contribution qr, where $1 \le r \le d(q) \le q$. Hence $qr = O(q^2) = o(q^2 \log q)$, and therefore (2.1) yields

$$L(n) \sim \frac{n^2}{2 \log n}.$$

For $n \ge 1$, since $2^{L(n)-1} \le a(n) < 2^{L(n)}$, we obtain the following consequence.

Corollary 5.1.

$$\log_2 a(n) \sim \frac{n^2}{2 \log n}.$$

6. Discussion and an open problem

The bit-length formula of Theorem 2.2 replaces a search over integers by an arithmetic identity: it ties the binary lengths of A351015 to the partial sums of the sorted multiplication table A061017, and through the divisor summatory estimate it fixes the growth $\log_2 a(n) \sim n^2/(2 \log n)$. What it does not settle is the least integer of that length. Proposition 3.1 reduces each minimizer to a choice of r boundary types and an ordering of the runs, and the algorithm of Section 3 carries out that selection by an exponential search: one bitmask dynamic program for each of the $\binom{d(q)}{r}$ boundary choices.

Problem 6.1. Compute $a(n)$ in time polynomial in n: construct the least binary word of length $L(n)$ with the run-type multiset of Proposition 3.1 directly, choosing the boundary types and the run order without a subset-indexed dynamic program.

The direct argument for $a(10)$ above, which fixes the entire ordering by a few greedy comparisons, suggests that such a construction may exist.

7. Reproducibility

The accompanying Python program `a351015_exact.py` implements the exact dynamic program of Section 3 and reproduces the table through $a(17)$. The C++17 program `a351015_fast_exact.cpp` applies the same recurrence with packed integer states and reproduces $a(17)$ through $a(20)$. A separate

standard-library Python program, `a351015_independent_verify.py`, imports neither producer. It treats these claimed words as certificates and exhaustively tests their prefixes for a smaller completable word across every minimum-cost boundary choice. Thus it checks $a(17)$ through $a(20)$ without repeating the producing recurrence. Together the programs reproduce the full table, which agrees with OEIS A351015 through $a(9)$. In the arXiv source they are ancillary files under `anc/`; the runs are invoked by

```
python anc/a351015_exact.py --max-n 17,
g++ -O3 -std=c++17 anc/a351015_fast_exact.cpp -o a351015_fast,
./a351015_fast,
python anc/a351015_independent_verify.py.
```

Adding `--details` to the Python command prints the binary words and run-type orderings; the C++ program prints these certificates by default.

Acknowledgments

The author thanks Gus Wiseman for creating OEIS A351015.

References

[1] G. Wiseman, *Smallest k such that the k-th composition in standard order has n distinct runs*, OEIS A351015, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A351015.

[2] A. Arnold, *Triangle read by rows, in which row n lists the compositions of n in reverse lexicographic order*, OEIS A066099, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A066099.

[3] J. Allen, *List in which n appears $d(n)$ times*, OEIS A061017, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A061017.

[4] N. J. A. Sloane, $a(n) = \sum_{k=1}^{n} \lfloor n/k \rfloor$; *also* $\sum_{k \le n} d(k)$, OEIS A006218, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A006218.

[5] G. Wiseman, *The smallest number whose binary expansion has exactly n distinct runs*, OEIS A350952, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A350952.

[6] G. Wiseman, *Number of integer compositions of n with all distinct runs*, OEIS A351013, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A351013.

[7] G. Wiseman, *Number of distinct runs in the n-th composition in standard order*, OEIS A351014, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A351014.

[8] G. Wiseman, *Numbers k such that the k-th composition in standard order has all distinct runs*, OEIS A351290, The On-Line Encyclopedia of Integer Sequences. https://oeis.org/A351290.

[9] H. S. Wilf, *The distribution of run lengths in integer compositions*, Electron. J. Combin. **18** (2011), no. 2, Paper P23. https://doi.org/10.37236/2019.

[10] A. Gafni, *Longest run of equal parts in a random integer composition*, Discrete Math. **338** (2015), no. 2, 236–247. https://doi.org/10.1016/j.disc.2014.10.003.

[11] G. H. Hardy and E. M. Wright, *An Introduction to the Theory of Numbers*, 6th ed., Oxford University Press, Oxford, 2008.

Email address: daniyal.kadirbekov@gmail.com